MTH 4300
Spring 2026

Programming Problem Set 7: Linked Lists – `linkedlist.cpp`

Directions: Download the template files provided on Brightspace. Submit your source code via Gradescope, in `.cpp` format.

Submission Reminder: Submitting for feedback is optional. If you submit by the deadline and want feedback beyond the autograder, indicate that in your submission. You may resubmit unlimited times.
Important: Passing all test cases does not guarantee correctness. The autograder tests limited inputs. Test additional cases yourself.
Note: While learning, avoid IDE tools that autocomplete or generate code for you (e.g., Copilot, IntelliSense suggestions). Writing code yourself builds the understanding you'll need for exams.

Working with AI on This Assignment

Step 1: Write your code on your own first. Give each part a genuine attempt before using any AI. If your code compiles and runs but gives wrong answers, try tracing through it with a specific input on paper before moving on to Step 2.

Step 2: If you're stuck, use AI as a debugging tutor. Don't paste your code and ask "what's wrong." Instead, use a prompt like this:

> *"I'm writing a C++ class called LinkedList that [describe what it should do]. Here is my code. I'm getting [wrong output / crash / compiler error]. Don't fix my code or show me corrected code. Instead, pick a specific line and ask me what I think a particular variable or expression evaluates to at that moment. Help me find the bug by making me trace through my own logic. Keep a back-and-forth conversation."*

Be sure to read the SPECIFICATIONS carefully! And write comments!

General Specifications and Notes:

- No global variables should be used.
- The only header allowed is `iostream`.
- You cannot use vectors or any advanced data structure that can store an arbitrarily long collection of numbers.
- Note: there is no user input.
- Remember to use `const` whenever appropriate.
- Include only the class definition (and the free-function definition of `operator< <`) in your submission — no `main` function.

Note: Each problem has test cases on Gradescope. All test cases are visible, so you can see exactly which cases your code passes or fails. Use this feedback to improve your solution. You may resubmit as many times as you like. Remember, these assignments are for practice; use them to prepare for exams!

The starter file `linkedlist.cpp` provides the `LinkedList` class as we developed it in class, with the constructor, destructor, and `insert(int n)` (tail-insert) method already implemented. You will extend the class by adding the methods and operators described below.

MTH 4300 S26

7a) **Methods**

Add the following methods to your `LinkedList` class:

- `int length() const` — returns the number of nodes in the list. Returns 0 for an empty list.
- `int get(int index) const` — returns the value stored at the node with the given index (zero-based). You may assume $0 \leq$ `index` $<$ `length()`.
- `void insertAt(int index, int value)` — inserts a new node containing `value` at position `index`. You may assume $0 \leq$ `index` $\leq$ `length()`. If `index` equals `length()`, the new node goes at the end.
- `void insertList(const LinkedList& other, int start)` — inserts a copy of `other` into this list so that the first element of `other`'s copy ends up at position `start`. You may assume $0 \leq$ `start` $\leq$ `length()`. The list `other` should be left unchanged. If `other` is empty, do nothing.

  Important: You should create new nodes (i.e., a deep copy of `other`) rather than reusing `other`'s existing nodes. Otherwise, both lists would point to the same nodes and the destructor would try to free the same memory twice.

  For example, if this list contains `1 -> 2 -> 3 -> 4 -> 5 -> nullptr`, and `other` contains `7 -> 8 -> nullptr`, and `start` is 2, then after the call this list should be

  `1 -> 2 -> 7 -> 8 -> 3 -> 4 -> 5 -> nullptr`.
- Overload the `< <` operator (as a friend function) so that `std::cout < < list;` prints the list in the format `1 -> 2 -> 3 -> nullptr` (with no newline at the end). An empty list should print as just `nullptr`.

7b) **The Rule of Three**

Add the following to your `LinkedList` class:

- **Copy constructor**: `LinkedList(const LinkedList& other)` — creates a new list that is a deep copy of `other`.
- **Copy assignment operator**: `LinkedList& operator=(const LinkedList& other)` — replaces this list's contents with a deep copy of `other`'s contents. Be sure to (1) handle the case where `other` is the same object as `*this`, (2) free this list's existing nodes before copying, and (3) return `*this` by reference.

Why are the copy constructor and copy assignment operator needed? Without them, C++ would generate default versions that simply shallow-copy the `head` pointer. Both lists would then point to the same chain of nodes, and when one is destroyed, the other would be left holding dangling pointers (and the destructor would try to free already-freed memory). Together with the destructor (which you already have), these three special member functions are sometimes called the *rule of three*: if you write any one of them for a class that manages heap memory, you almost always need to write all three.

You will submit this part in a separate submission area on Gradescope (the same `linkedlist.cpp` file, now with the above additions included).

7c) **Practice Questions**

The following are extra practice questions for you. They are not graded, but they are good preparation for exams. Try them on your own, and let me know if you have questions.

1. In your `insertList`, you copied `other`'s values into newly allocated nodes. An alternative approach (sometimes called *splicing*) would be to rewire `other`'s existing nodes directly into this list's chain, with no new allocation. Sketch how this alternative would work. What goes wrong if `other` is destroyed after the operation? Why does the `const LinkedList& other` parameter rule out this alternative?
2. Write a method `void reverse()` that reverses the list in place (i.e., the head becomes the tail and vice versa). *Hint:* you will need three pointers — one for the previous node, one for the current node, and one for the next node — as you walk through the list. Sketch the pointer changes for a 3-node list before writing any code. What are the edge cases?
3. Overload the == operator so that two lists compare equal if and only if they have the same length and the same value at every index. You may implement this as a member function or as a free function. *Hint:* walk both lists in parallel.
4. Overload the += operator so that `list1 += list2` appends a copy of `list2` to the end of `list1`. The operator should return `*this` by reference, so that chained operations like `a += b += c` work. *Hint:* this is closely related to your `insertList` method.